Tests unitaires et TDD

Objectifs

- **Comprendre** les fondements des tests unitaires et leur rôle dans la qualité logicielle
- **Écrire** des tests unitaires efficaces : logique métier, validation d'entrées, cas limites, erreurs, valeur de retour, état et interactions
- **Appliquer** les méthodologies TDD et BDD
- **Automatiser** l'exécution des tests dans l'intégration continue
- **Assurer** une couverture suffisante et éviter les pièges courants (tests fragiles, faux positifs)
- **Analyser** les résultats des tests et interpréter les erreurs

MODULE 01

Fondamentaux des tests unitaires et TDD

C'est quoi la qualité logicielle ?

Comment définir la qualité d'un logiciel ?

Qualité logicielle

- **Fonctionnalité** : faire ce qui est attendu
- **Fiabilité** : le faire sans erreur, dans la durée
- **Maintenabilité** : pouvoir le faire évoluer à coût raisonnable
- **Performance**, **sécurité**, **utilisabilité**…

Pourquoi s'en préoccuper ?

La qualité ou plus exactement le manque de qualité coute cher en terme d'argent, de réputation, d'ambiance de travail, etc.

Quels acteurs ?

Même si le code est produit par des développeurs, il est nécessaire de comprendre la qualité logicielle comme un résultat **collectif** nécessitant l'implication et le support de tous les acteurs : du client, au testeur, en passant par le manager jusqu'au développeur.

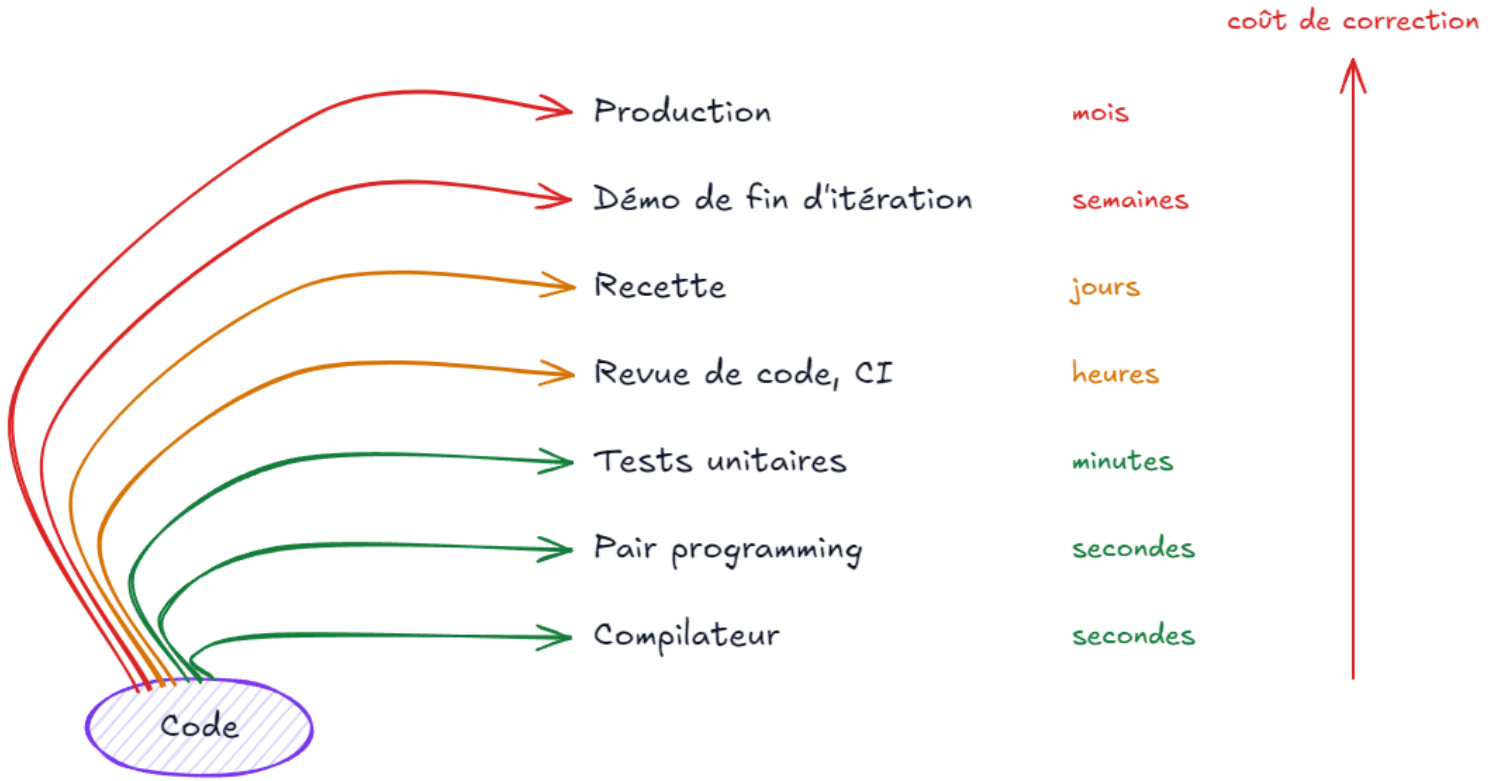
Quels leviers ?

La qualité logicielle est une démarche continue propre à chaque contexte passant aussi bien par les processus organisationnels que par les processus opérationnels. Elle peut être, bien entendue, facilité par l'utilisation d'outils adéquats comme des IDEs, des usines logiciels mais est aussi affaire de pratiques, d'apprentissage, d'entrainement et donc de temps de montée en compétences. C'est pourquoi le **Clean Code**, le **Craft**, le **TDD**, le **DDD** sont des boites à outils utiles mais pas auto-suffisantes.

Un point sur les métriques

La qualité logicielle étant une démarche continue, il est intéressant de pouvoir suivre et mesurer l'évolution de celle-ci au cours du temps via des métriques telles que le code coverage, le pourcentage de duplication, le nombre de ligne de code, etc tout en gardant à l'esprit que ces métriques ne sont pas absolues.

Boucle de feedback



- Plus la boucle de feedback est **courte**, moins un bug coûte

C’est quoi un test ?

Action manuelle ou automatisée vérifiant dans un contexte donné qu'un bloc code a les effets attendus au regard d'un exemple d'un concept particulier.

Pourquoi tester ?

Ecrire des tests automatisés offre plusieurs avantages :

- Valider que le code fait ce que l'on pense
- Faciliter la détection des régressions
- Faciliter le refactoring
- Documenter le comportement attendu par l'exemple

Le vrai objectif

> Permettre une croissance **durable** du projet logiciel.
>
> — Vladimir Khorikov, *Unit Testing Principles, Practices, and Patterns*

- Sans tests, la vitesse de développement **chute** avec le temps
- Un test a un **coût** : écriture, maintenance, exécution
- Seuls les tests dont la **valeur** dépasse le coût sont utiles
- ⚠️ Un mauvais test est pire que pas de test

Les familles de tests



Les familles de tests

- **Unitaire** : une unité de comportement, en isolation, en mémoire
- **Intégration** : plusieurs composants connectés, souvent avec une vraie dépendance (base de données, fichier…)
- **End-to-end** : le système complet, piloté comme un utilisateur (IHM, API)
- Plus on monte : plus de **confiance** sur l'ensemble, mais plus **lent**, **fragile** et **coûteux**
- ⚠️ La taxonomie varie selon les équipes : s'accorder sur un **vocabulaire commun**

Les propriétés FIRST

- **Fast** : rapide, pour être lancé en permanence
- **Independent** : aucun ordre, aucun état partagé entre tests
- **Repeatable** : même résultat partout, à chaque fois (pas d'horloge, de réseau, de hasard non maîtrisés)
- **Self-validating** : succès ou échec sans interprétation humaine
- **Timely** / **Thorough** : écrit au bon moment, couvre cas nominaux et cas à la marge

Comment je m’y prends ?

Le pattern des 3A :

- **ARRANGE** : code permettant de mettre en place le contexte pour le code à tester.
  - *Exemple :* Etant donné A égal à 10 et B égal à 2.
- **ACT** : code exécutant l’action qu’on cherche à valider.
  - *Exemple :* Quant on divise A par B.
- **ASSERT** : code vérifiant les effets attendus du code testé.
  - *Exemple :* Alors on doit obtenir 2.

Pourquoi AAA ?

> Arrange-Act-Assert est un concept simple […] mais il ne coûte rien à pratiquer et offre une façon cohérente de communiquer.
>
> — Tim Ottinger et Jeff Langr

- Pattern nommé par **Bill Wake**
- Le code est surtout **lu** : par les collègues, par soi-même dans six mois
- Repérer les 3 parties d'un coup d'oeil permet de comprendre le test rapidement

Anatomie d'un test

Nom : le scénario

Arrange : le contexte

Act : une seule action

Assert : l'effet attendu

```
test('remise de 10 % au-delà de 100 €', () => {
  const cart = cartWithItemsWorth(120);
                                          ← ligne vide
  const total = cart.total();
                                          ← ligne vide
  expect(total).toBe(108);
});
```

Règles d'un test bien structuré

- **Un seul Act** : une ligne, un comportement ; plusieurs actes = plusieurs tests
- **Pas de** `if` ni de boucle dans un test
- **Arrange** souvent le plus long : le factoriser (builders, fonctions)
- **Assert** : plusieurs assertions acceptées si elles décrivent **un même** comportement
- Nommer le résultat `result` / `total` / `sut` pour le repérer

Ok mais je vérifie quoi ?

La vérification des effets attendus s'effectue des exemples de deux types de cas d'utilisation :

- **Cas nominal** : cas où les exemples d'utilisation correspondent aux cas généraux (happy path).
  - *Exemple :* Prenons le cas de la division, des exemples de cas généraux sont 1/2, 3/7, etc.
- **Cas à la marge** : cas où les exemples d'utilisation correspondent à des erreurs, des exceptions, des cas non généraux.
  - *Exemple :* Prenons le cas de la division, un exemple de cas à la marge est la division par 0.

Dans le cadre des user stories, les ateliers collaboratifs de type Example Mapping peuvent être un bon point d'entrée pour définir des exemples concrets de vos critères d'acceptances.

Les outils nécessaires

- **Bibliothèque de tests** : déclarer des tests et des suites
- **Bibliothèque d'assertions** : exprimer les attendus
- **Test runner** : découvrir, exécuter, rapporter

À retenir

- Un bon test **protège** des régressions et **résiste** au refactoring
- Structure **AAA**, un seul Act par test, nom **métier**
- **FIRST** : rapides, indépendants, déterministes
- Tester règles métier, validations et **bornes**

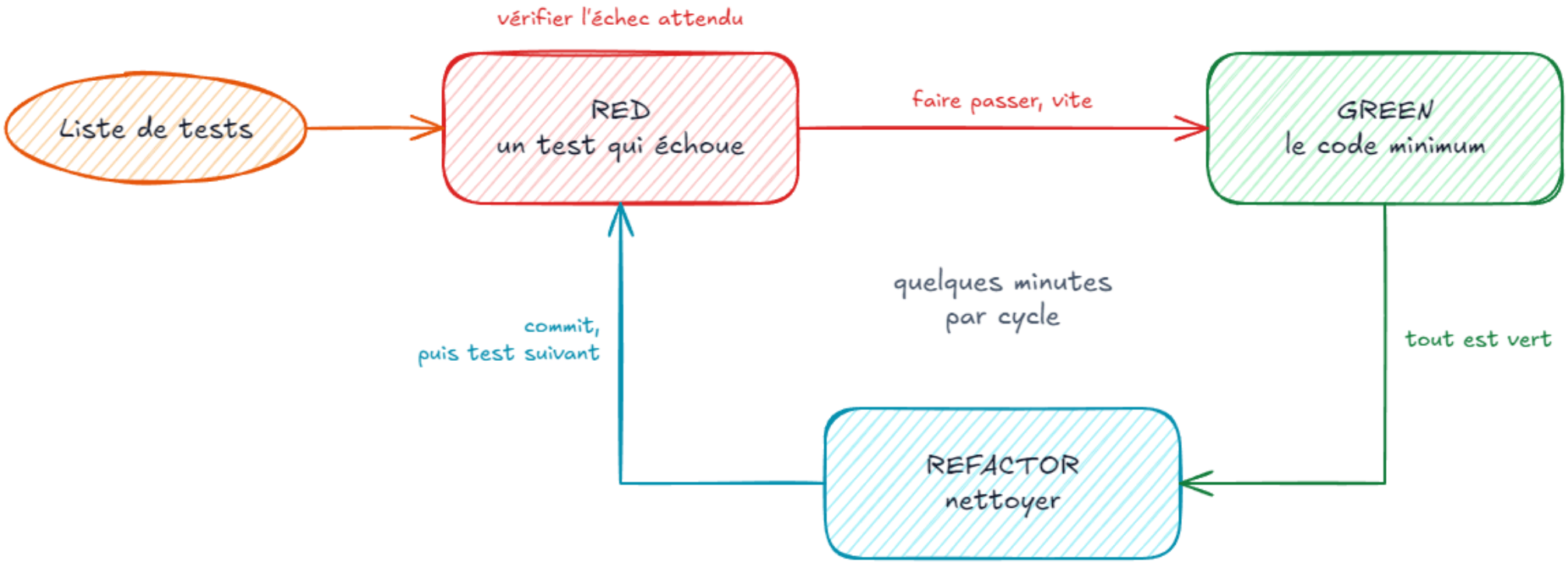
MODULE 02

TDD

Quand écrire les tests ?

- **Test after** : code d'abord, tests ensuite
  - souvent piloté par la couverture, tests couplés à l'implémentation
  - tests parfois oubliés faute de temps
- **Test first** : tests d'abord, code ensuite
  - l'attendu est clarifié avant l'implémentation
- **TDD** : test first **par petites itérations** avec refactoring systématique

Le cycle TDD


Liste de tests
vérifier l'échec attendu
RED
un test qui échoue
faire passer, vite
GREEN
le code minimum
quelques minutes
par cycle
commit,
puis test suivant
tout est vert
REFACTOR
nettoyer

Red – Green – Refactor

- 🔴 **Red** : écrire **un** test qui échoue (ou ne compile pas)
  - lire l'échec : **rouge pour la bonne raison** (valeur inattendue plutôt qu'exception)
- 🟢 **Green** : écrire le code **minimum** pour le faire passer, vite
- 🔵 **Refactor** : nettoyer code et tests, sans changer le comportement
- 🔁 Recommencer, par cycles de quelques minutes ; **commit** au vert
- TDD est une technique de **conception** : les tests sont une conséquence

Les 3 lois du TDD

1. Ne pas écrire de code de production **sans un test qui échoue**
2. Ne pas écrire plus de test que nécessaire pour **échouer** (ne pas compiler compte)
3. Ne pas écrire plus de code de production que nécessaire pour **passer** le test

— Robert C. Martin

Les patterns du TDD

- **Baby steps** : la plus petite étape possible, surtout quand ça devient difficile
- **Fake it ('til you make it)** : retourner une constante pour passer au vert
- **Triangulation** : ajouter un exemple pour forcer la généralisation
- **Obvious implementation** : écrire directement la solution quand elle est évidente
- 🚦 Revenir aux baby steps dès qu'un test reste rouge trop longtemps

FizzBuzz : les règles

- Multiple de 3 : `Fizz`
- Multiple de 5 : `Buzz`
- Multiple de 3 et de 5 : `FizzBuzz`
- Sinon : le nombre lui-même

Liste de tests :

- ☐ `1` → `"1"`, `2` → `"2"`
- ☐ `3` → `"Fizz"`, `6` → `"Fizz"`
- ☐ `5` → `"Buzz"`, `10` → `"Buzz"`
- ☐ `15` → `"FizzBuzz"`

FizzBuzz : itération 1

🔴 Red

```
import { fizzBuzz } from './fizz-buzz';

it('retourne le nombre lui-même', () => {
  expect(fizzBuzz(1)).toBe('1');
});
```

🟢 Green : fake it

```
export function fizzBuzz(n: number): string {
  return '1';
}
```

FizzBuzz : itération 2

🔴 **Red** : triangulation

```
it.each([
  [1, '1'],
  [2, '2'],
])('retourne le nombre lui-même : %i', (n, expected) => {
  expect(fizzBuzz(n)).toBe(expected);
});
```

🟢 **Green** : la constante ne suffit plus

```
export function fizzBuzz(n: number): string {
  return String(n);
}
```

- Exemples similaires **regroupés** : plus d'exemples aident le lecteur à retrouver la règle

FizzBuzz : itération 3

🔴 Red

```
it('retourne Fizz pour 3', () => {
  expect(fizzBuzz(3)).toBe('Fizz');
});
```

🟢 Green : minimum, même naïf

```
export function fizzBuzz(n: number): string {
  if (n === 3) return 'Fizz';
  return String(n);
}
```

FizzBuzz : itération 4

🔴 **Red** : triangulation, 3 exemples interdisent un 3e `if` (règle de trois)

```
it.each([3, 6, 9])('retourne Fizz pour un multiple de 3 : %i', (n) => {
  expect(fizzBuzz(n)).toBe('Fizz');
});
```

🟢 **Green**

```
export function fizzBuzz(n: number): string {
  if (n % 3 === 0) return 'Fizz';
  return String(n);
}
```

FizzBuzz : itération 5

🔴 **Red** et 🟢 **Green** : même démarche pour `Buzz`

```
export function fizzBuzz(n: number): string {
  if (n % 3 === 0) return 'Fizz';
  if (n % 5 === 0) return 'Buzz';
  return String(n);
}
```

🔵 **Refactor** : révéler l'intention

```
const isMultipleOf = (n: number, divisor: number) => n % divisor === 0;
```

FizzBuzz : itération 6

🔴 Red

```
it.each([15, 30])('retourne FizzBuzz pour un multiple de 15 : %i', (n) => {
  expect(fizzBuzz(n)).toBe('FizzBuzz');
});
```

🟢 Green puis 🔵 Refactor

```
export function fizzBuzz(n: number): string {
  if (isMultipleOf(n, 15)) return 'FizzBuzz';
  if (isMultipleOf(n, 3)) return 'Fizz';
  if (isMultipleOf(n, 5)) return 'Buzz';
  return String(n);
}
```

Ce qu'apporte le TDD

- 🎯 **Clarifie** l'objectif avant d'implémenter
- 👣 **Avance** par petits incréments, toujours au vert
- 😌 **Réduit le stress** : une seule chose à la fois, le reste est dans la liste
- ✂️ **Évite** l'over-engineering ; 🧩 **pousse** vers un code découplé
- 🛡️ **Produit** un filet de sécurité et une documentation vivante
- ⚠️ Demande de l'**entraînement** : katas, pair programming

L'étape Refactor

- Modifier la **structure** sans modifier le **comportement**
- Uniquement **au vert** : les tests sont le filet de sécurité
- Par **petits pas**, en relançant les tests à chaque pas
- Concerne **aussi les tests**
- Supprimer la **duplication**, améliorer la **lisibilité**, traiter les **code smells**

> Always leave the campground cleaner than you found it.
>
> — Boy Scout Rule

À retenir

- **TDD** : Red – Green – Refactor, petits pas
- Une barre verte est une **permission de penser**
- Tester règles métier, validations et **bornes**

MODULE 03

Mocks, tests avancés

Tester les erreurs

- Les erreurs font partie du **contrat** d'une fonction, au même titre que la valeur de retour
- Vérifier **quelle** erreur, pas seulement « ça plante »
- Cas typiques : entrée invalide, règle métier violée, dépendance en échec
- Une erreur non testée finit souvent en **500** en production

Une erreur métier

```
export class InsufficientFundsError extends Error {
  constructor(readonly balance: number, readonly amount: number) {
    super(`Solde insuffisant : ${balance} € pour un retrait de ${amount} €`);
    this.name = 'InsufficientFundsError';
  }
}

export class BankAccount {
  constructor(private balance: number) {}

  withdraw(amount: number): void {
    if (amount <= 0) throw new RangeError('Le montant doit être positif');
    if (amount > this.balance) throw new InsufficientFundsError(this.balance, amount);
    this.balance -= amount;
  }
}
```

toThrow

```
it('refuse un retrait supérieur au solde', () => {
  const account = new BankAccount(100);

  expect(() => account.withdraw(150)).toThrow(InsufficientFundsError);
});

it('indique le solde dans le message', () => {
  const account = new BankAccount(100);

  expect(() => account.withdraw(150)).toThrow('Solde insuffisant : 100 €');
});
```

- ⚠️ Passer une **fonction** à `expect` : `expect(account.withdraw(150))` lève avant l'assertion
- `toThrow(Classe)`, `toThrow('extrait du message')`, `toThrow(/regex/)`

Vérifier le contenu de l'erreur

```
it('précise le solde et le montant demandé', () => {
  const account = new BankAccount(100);

  expect(() => account.withdraw(150)).toThrow(
    expect.objectContaining({ balance: 100, amount: 150 }),
  );
});

it.each([0, -10])('refuse un montant de %i €', (amount) => {
  const account = new BankAccount(100);

  expect(() => account.withdraw(amount)).toThrow(RangeError);
});
```

- Un type d'erreur par **cas métier** : plus simple à tester et à traiter

Erreurs asynchrones

```
it('refuse de valider un panier vide', async () => {
  const checkout = new Checkout(new StubPaymentGateway());

  await expect(checkout.confirm(emptyCart)).rejects.toThrow(EmptyCartError);
});
```

```
// ❌ Sans await : le test se termine avant la promesse et passe quand même
it('refuse de valider un panier vide', () => {
  expect(checkout.confirm(emptyCart)).rejects.toThrow(EmptyCartError);
});
```

- `rejects` / `resolves` + **`await`** (ou `return`)

Alternative : retourner l'erreur

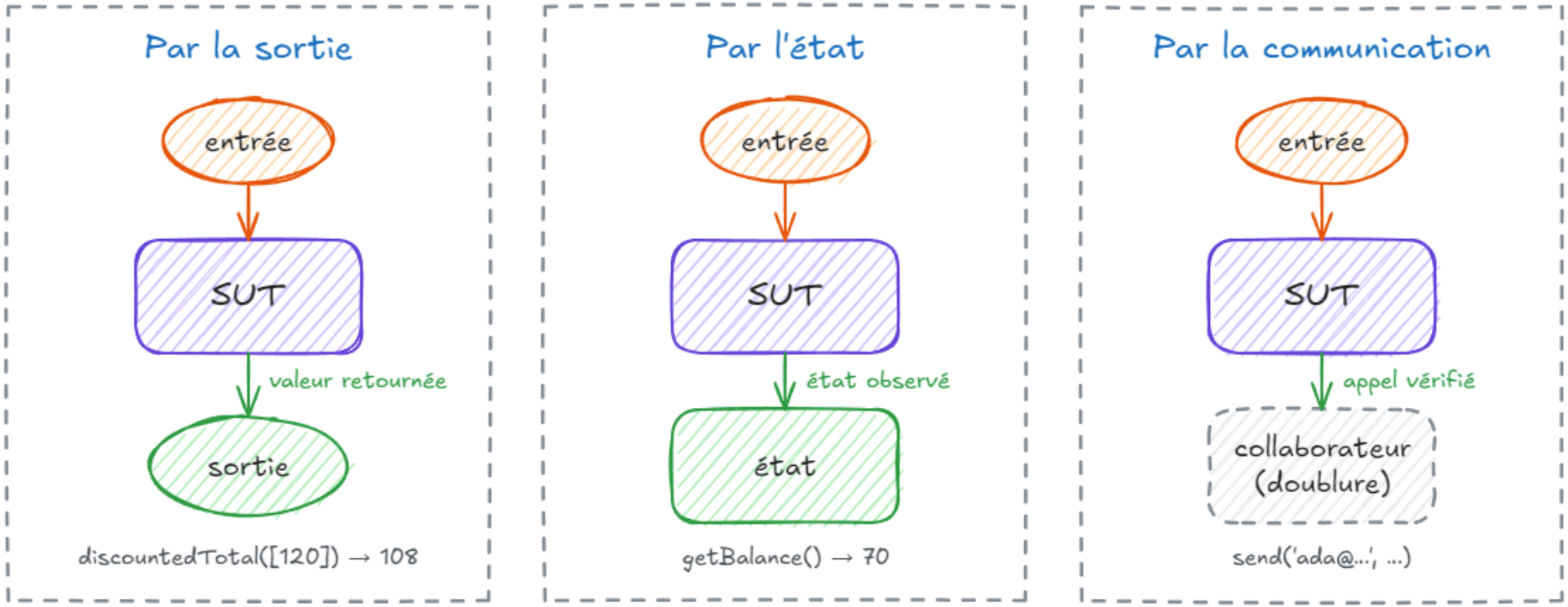
```ts
type Result<T, E> = { ok: true; value: T } | { ok: false; error: E };

function parseAge(input: string): Result<number, string> {
  const age = Number(input);
  if (!Number.isInteger(age) || age < 0) return { ok: false, error: 'Âge invalide' };
  return { ok: true, value: age };
}

it('refuse un âge négatif', () => {
  expect(parseAge('-3')).toEqual({ ok: false, error: 'Âge invalide' });
});
```

- Erreur **attendue** dans le type de retour : test par la sortie, plus simple
- Exceptions pour l'**inattendu**

Trois styles de test



Test par la sortie

```
it('applique 10 % de remise au-delà de 100 €', () => {
  expect(discountedTotal([120])).toBe(108);
});
```

- **Output-based** : on fournit une entrée, on vérifie la valeur **retournée**
- Seulement pour du code **sans effet de bord** (fonction pure)
- Le plus **lisible**, le plus **robuste** au refactoring

Test par l'état

```
it('débite le compte du montant retiré', () => {
  const account = new BankAccount(100);

  account.withdraw(30);

  expect(account.getBalance()).toBe(70);
});
```

- **State-based** : on agit, puis on vérifie l'**état** du système ou d'une dépendance
- Lire l'état via l'**API publique**, jamais via un champ privé
- ⚠️ Ne pas exposer d'état **uniquement** pour le test

Test par la communication

```
it('envoie un email de confirmation', () => {
  const emailSender = { send: jest.fn() };
  const orderService = new OrderService(emailSender);

  orderService.confirm(order);

  expect(emailSender.send).toHaveBeenCalledWith('ada@example.com', 'Commande A-42 confirmée');
});
```

- **Communication-based** : on vérifie les **appels** faits à un collaborateur
- Nécessite une **doublure** (ici un mock)
- Utile quand l'appel **est** le comportement attendu (email, paiement, message)

Quel style préférer ?

	Sortie	État	Communication
Résistance au refactoring	✅ forte	🟡 moyenne	🟠 faible
Maintenabilité	✅ peu de setup	🟡 moyenne	🟠 doublures à configurer
Code concerné	fonctions pures	objets à état	effets de bord

- Préférer **sortie > état > communication** — Vladimir Khorikov
- Pousser la logique vers du code **pur** pour multiplier les tests par la sortie

Les dépendances cachées

```
export class LunchService {
  isLunchTime(): boolean {
    const hour = new Date().getHours();
    return hour >= 12 && hour < 14;
  }
}
```

- `new Date()` est une dépendance **implicite** et non maîtrisée
- Le test passe à 13 h, échoue à 15 h : **non déterministe**
- Même problème : `Math.random()`, `fetch`, `process.env`, système de fichiers
- Solution : rendre la dépendance **explicite** et **contrôlable** par le test

Passage par valeur

```
export class LunchService {
  isLunchTime(now: Date): boolean {
    const hour = now.getHours();
    return hour >= 12 && hour < 14;
  }
}

it("c'est l'heure du déjeuner à 13 h 30", () => {
  const service = new LunchService();

  expect(service.isLunchTime(new Date(2026, 0, 1, 13, 30))).toBe(true);
});
```

- Le plus **simple** : la fonction devient pure
- La décision remonte à l'appelant

Passage par fonction

```
export class LunchService {
  constructor(private readonly now: () => Date = () => new Date()) {}

  isLunchTime(): boolean {
    const hour = this.now().getHours();
    return hour >= 12 && hour < 14;
  }
}

it("ce n'est pas l'heure du déjeuner à 15 h", () => {
  const service = new LunchService(() => new Date(2026, 0, 1, 15, 0));

  expect(service.isLunchTime()).toBe(false);
});
```

- Léger, **valeur par défaut** pour le code de production

Passage par contrat

```ts
export interface Clock { now(): Date }

export class LunchService {
  constructor(private readonly clock: Clock) {}

  isLunchTime(): boolean {
    const hour = this.clock.now().getHours();
    return hour >= 12 && hour < 14;
  }
}

const clockAt = (hour: number): Clock => ({ now: () => new Date(2026, 0, 1, hour, 0) });

it("c'est l'heure du déjeuner à 12 h", () => {
  expect(new LunchService(clockAt(12)).isLunchTime()).toBe(true);
});
```

Surcharge par héritage

```
export class LunchService {
  isLunchTime(): boolean {
    const hour = this.now().getHours();
    return hour >= 12 && hour < 14;
  }

  protected now(): Date {
    return new Date();
  }
}

class LunchServiceAt extends LunchService {
  constructor(private readonly fixed: Date) { super(); }

  protected override now(): Date { return this.fixed; }
}
```

- Utile sur du **legacy** (seam), mais couple le test aux détails internes

Comparatif

Technique	Quand l'utiliser
Valeur	logique pure, appelant qui connaît la donnée
Fonction	une seule opération à remplacer
Contrat (interface)	collaborateur riche, plusieurs implémentations
Héritage	code legacy, sans toucher la signature

- Dans tous les cas : **injection de dépendances** plutôt que création interne
- Les dépendances visibles dans le constructeur **documentent** la classe

Inversion de dépendances

```
export interface EmailSender {
  send(to: string, subject: string): Promise<void>;
}

export class OrderService {
  constructor(private readonly emailSender: EmailSender) {}
  // ...
}

// Composition root : seul endroit qui connaît les implémentations
const orderService = new OrderService(new SmtpEmailSender(config.smtp));
```

- **DIP** : le métier dépend d'une **abstraction**, pas de l'infrastructure
- L'abstraction est définie **du point de vue du métier**

SOLID et testabilité

Principe	Effet sur les tests
Single Responsibility	une raison de changer = moins de cas par classe
Open/Closed	ajouter un comportement sans casser les tests existants
Liskov Substitution	une doublure doit respecter le contrat
Interface Segregation	petites interfaces = doublures simples
Dependency Inversion	dépendances injectables et remplaçables

- Un code **difficile à tester** révèle souvent un problème de conception

Cœur fonctionnel, coquille impérative

```
// Cœur : logique pure, testée par la sortie
export function shippingCost(order: Order): number {
  if (order.customer.isPremium() || order.total() >= 50) return 0;
  return 4.9;
}

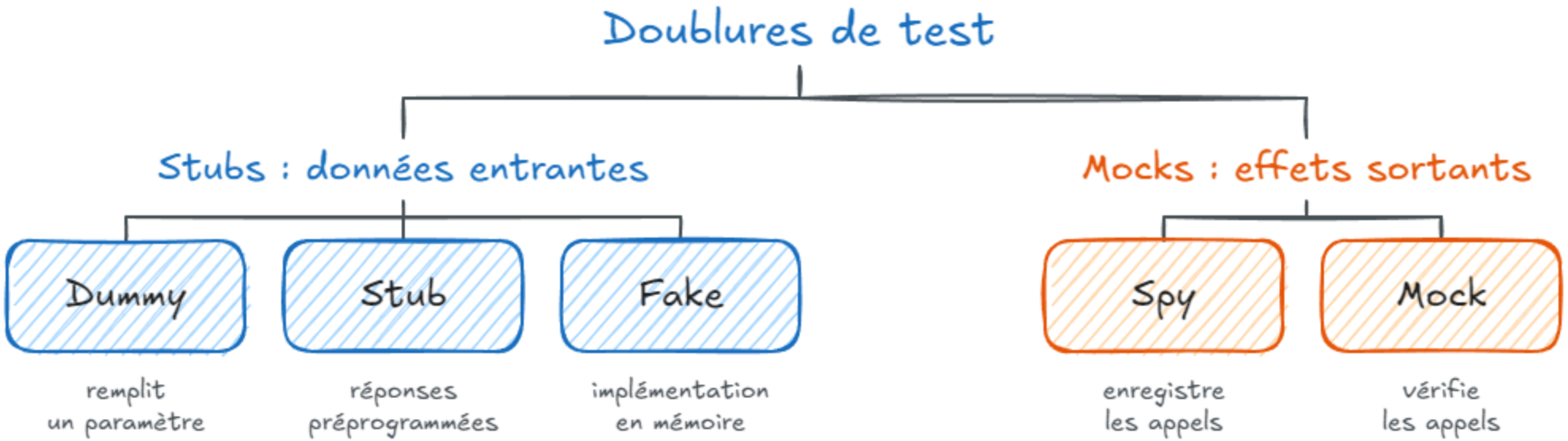
// Coquille : I/O sans logique (humble object), testée en intégration
export async function applyShippingCost(orderId: string, db: Database) {
  const order = await db.loadOrder(orderId);
  await db.updateShippingCost(orderId, shippingCost(order));
}
```

- Isoler les **décisions** des **effets de bord**
- La coquille est si simple qu'elle a peu besoin de tests unitaires

Pourquoi des doublures ?

- 🐢 **Lent** : base de données, réseau, fichiers
- 🎲 **Non déterministe** : horloge, hasard, service distant
- 🚧 **Indisponible** : API tierce en cours de développement, environnement absent
- 💥 **Effets de bord** : envoi d'email, paiement, message publié
- 🔬 **Cas difficiles à provoquer** : timeout, erreur 503, disque plein

Les doublures de test



Les doublures de test

Doublure	Rôle
Dummy	remplit un paramètre, n'est jamais utilisé
Stub	fournit des réponses préprogrammées
Spy	enregistre les appels pour les vérifier ensuite
Mock	vérifie les appels attendus
Fake	implémentation simplifiée mais fonctionnelle (en mémoire)

- Vocabulaire de Gerard Meszaros (*xUnit Test Patterns*)
- `jest.fn()` peut jouer stub, spy ou mock : c'est l'**usage** qui compte

Dummy et stub

```
const dummyLogger: Logger = {
  log: () => { throw new Error('ne doit pas être appelé'); },
};

const stubRates: ExchangeRates = { rateFor: () => 1.1 };

it('convertit un prix en dollars', () => {
  const converter = new PriceConverter(stubRates, dummyLogger);

  expect(converter.toUsd(100)).toBeCloseTo(110);
});
```

- **Dummy** : seulement pour satisfaire la signature
- **Stub** : alimente le SUT en **données entrantes**

Spy et mock

```
class EmailSenderSpy implements EmailSender {
  readonly sent: { to: string; subject: string }[] = [];

  async send(to: string, subject: string) {
    this.sent.push({ to, subject });
  }
}

it('envoie un email de confirmation', async () => {
  const emailSender = new EmailSenderSpy();

  await new OrderService(emailSender).confirm(order);

  expect(emailSender.sent).toEqual([{ to: 'ada@example.com', subject: 'Commande A-42 confirmée' }]);
});
```

- Vérifie les **interactions sortantes** ; avec Jest : `jest.fn()` + `toHaveBeenCalledWith`

Fake

```
export class InMemoryOrderRepository implements OrderRepository {
  private readonly orders = new Map<string, Order>();

  async save(order: Order): Promise<void> {
    this.orders.set(order.id, order);
  }

  async findById(id: string): Promise<Order | undefined> {
    return this.orders.get(id);
  }
}
```

- Se comporte **comme le vrai**, sans base de données
- Réutilisable dans de nombreux tests, **sans configuration**
- À maintenir : tester le fake et le vrai avec les **mêmes tests** de contrat

Mock ou stub ?


Stub
ExchangeRates
donnée entrante
rateFor('USD') → 1.1
SUT
effet sortant
send(to, subject)
Mock
EmailSender
ne pas vérifier l'appel
vérifier la sortie
vérifier l'appel

Ne jamais vérifier un stub

```
it('convertit un prix en dollars', () => {
  const rateFor = jest.fn().mockReturnValue(1.1);
  const converter = new PriceConverter({ rateFor }, dummyLogger);

  expect(converter.toUsd(100)).toBeCloseTo(110);
  // ❌ Détail d'implémentation : la sortie suffit
  expect(rateFor).toHaveBeenCalledWith('USD');
});
```

- Un stub sert à **alimenter** le SUT, pas à le vérifier
- Vérifier un appel entrant = **surspécification** : le test casse au refactoring
- Vérifier les appels **sortants** qui ont un effet visible de l'extérieur

Jest : configurer une doublure

```ts
const rateFor = jest.fn((currency: string) => 1);

rateFor.mockReturnValue(1.1);
rateFor.mockReturnValueOnce(0.9);
rateFor.mockImplementation((currency) => (currency === 'USD' ? 1.1 : 0.85));

const fetchRate = jest.fn(async (currency: string) => 1);

fetchRate.mockResolvedValue(1.1);
fetchRate.mockRejectedValueOnce(new Error('Service indisponible'));
```

- `mockReturnValue` : toujours ; `…Once` : au prochain appel seulement
- `mockResolvedValue` / `mockRejectedValue` : promesses

Jest : vérifier les appels

```
expect(emailSender.send).toHaveBeenCalled();
expect(emailSender.send).toHaveBeenCalledTimes(1);
expect(emailSender.send).toHaveBeenCalledWith('ada@example.com', expect.stringContaining('A-42'));
expect(paymentGateway.refund).not.toHaveBeenCalled();

expect(emailSender.send.mock.calls).toEqual([['ada@example.com', 'Commande A-42 confirmée']]);
```

- **Matchers asymétriques** : `expect.any(Date)`, `expect.stringContaining(…)`, `expect.objectContaining({ … })`
- Vérifier **juste ce qui compte** pour le comportement

Jest : `spyOn`

```
it('génère un code promo à 6 caractères', () => {
  const random = jest.spyOn(Math, 'random').mockReturnValue(0.42);

  const code = generateCouponCode();

  expect(code).toHaveLength(6);
  random.mockRestore();
});
```

- Remplace une méthode d'un **objet existant**, sans changer le code de production
- `mockRestore()` remet l'original (ou `restoreMocks: true` dans la config)
- Utile pour du code **non injectable** ; sinon préférer l'injection

Jest : `jest.mock`

```
import { sendEmail } from './mailer';
import { confirmOrder } from './order';

jest.mock('./mailer');

it('envoie un email de confirmation', async () => {
  jest.mocked(sendEmail).mockResolvedValue();

  await confirmOrder({ id: 'A-42', customerEmail: 'ada@example.com' });

  expect(sendEmail).toHaveBeenCalledWith('ada@example.com', 'Commande A-42 confirmée');
});
```

- Remplace **tout un module** par des `jest.fn()` automatiques
- ⚠️ Couple le test au **chemin d'import** : préférer l'injection quand c'est possible

Jest : fake timers

```
beforeEach(() => jest.useFakeTimers());
afterEach(() => jest.useRealTimers());

it('relance le paiement après 5 secondes', () => {
  const retryPayment = jest.fn();

  scheduleRetry(retryPayment);
  jest.advanceTimersByTime(5000);

  expect(retryPayment).toHaveBeenCalledTimes(1);
});
```

- `advanceTimersByTime` : exécuter les `setTimeout` / `setInterval` sans attendre
- `jest.setSystemTime(date)` : fixer `new Date()` et `Date.now()` pour du code non injectable

Jest : nettoyer entre les tests

Option	Effet
`mockClear()` / `clearMocks`	efface les appels enregistrés
`mockReset()` / `resetMocks`	idem + supprime les implémentations
`mockRestore()` / `restoreMocks`	idem + restaure l'original (`spyOn`)

```
// jest.config.js
module.exports = { preset: 'ts-jest', clearMocks: true, restoreMocks: true };
```

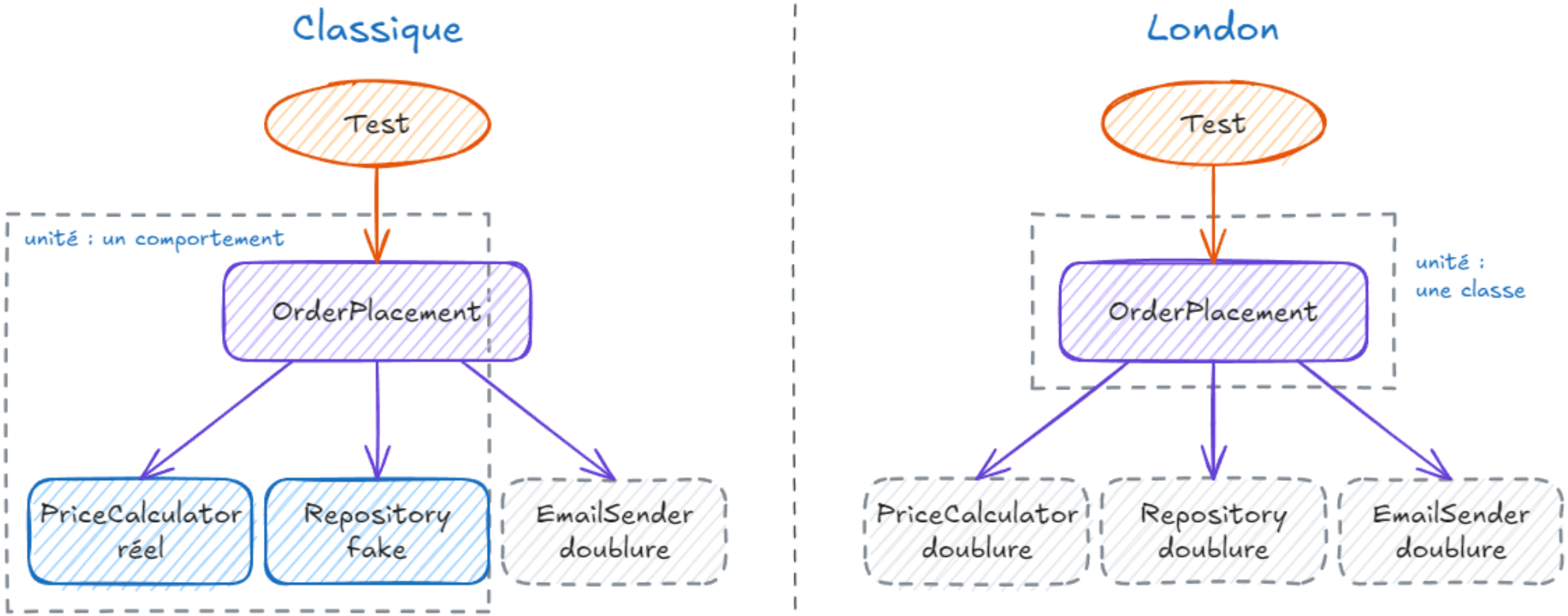
- Sans nettoyage, les appels d'un test **fuient** dans le suivant

Aide-mémoire

Besoin	Jest	Sinon
Créer une doublure	`jest.fn()`	`sinon.stub()` / `sinon.spy()`
Valeur de retour	`mockReturnValue(v)`	`.returns(v)`
Promesse	`mockResolvedValue(v)`	`.resolves(v)`
Remplacer une méthode	`jest.spyOn(obj, 'm')`	`sinon.stub(obj, 'm')`
Vérifier un appel	`toHaveBeenCalledWith(…)`	`sinon.assert.calledWith(…)`
Horloge	`jest.useFakeTimers()`	`sinon.useFakeTimers()`

- Java : **Mockito** ; Python : **unittest.mock** ; mêmes concepts

Deux écoles


Classique
Test
unité : un comportement
OrderPlacement
PriceCalculator
réel
Repository
fake
EmailSender
doublure
London
Test
OrderPlacement
unité :
une classe
PriceCalculator
doublure
Repository
doublure
EmailSender
doublure

Deux écoles

	Classique (Detroit)	London (mockist)
Unité	un **comportement**	une **classe**
Isolation	des **tests** entre eux	de la classe et de ses **collaborateurs**
Doublures pour	dépendances partagées, hors processus	tout collaborateur mutable
Style dominant	sortie, état	communication
Conception	souvent inside-out	outside-in
Références	Kent Beck, Vladimir Khorikov	Freeman et Pryce (*GOOS*)

École classique

```
it('confirme la commande et prévient le client', async () => {
  const repository = new InMemoryOrderRepository();
  const emailSender = new EmailSenderSpy();
  const service = new OrderPlacement(repository, new PriceCalculator(), emailSender);

  await service.place('A-42', 'ada@example.com', [{ label: 'Clavier', price: 120 }]);

  expect(await repository.findById('A-42')).toMatchObject({ total: 108 });
  expect(emailSender.sent).toHaveLength(1);
});
```

- Vrais collaborateurs en mémoire (`PriceCalculator`), fake pour le stockage
- Doublure seulement pour l'**effet externe** (email)

École London

```
it('enregistre la commande au prix calculé', async () => {
  const repository = { save: jest.fn(), findById: jest.fn() };
  const calculator = { total: jest.fn().mockReturnValue(108) };
  const emailSender = { send: jest.fn() };
  const service = new OrderPlacement(repository, calculator, emailSender);

  await service.place('A-42', 'ada@example.com', [{ label: 'Clavier', price: 120 }]);

  expect(repository.save).toHaveBeenCalledWith(expect.objectContaining({ total: 108 }));
});
```

- Chaque collaborateur est une doublure : on teste **une classe** et ses échanges
- Guide la conception des **interfaces** par l'usage

Forces et faiblesses

	Classique	London
✅	résistant au refactoring	échec localisé précisément
✅	tests proches du métier	setup simple, conception guidée
⚠️	un bug peut rendre rouges plusieurs tests	tests couplés à l'implémentation
⚠️	setup parfois plus lourd	refactoring = tests à réécrire

- Pas de dogme : choisir **en conscience**, souvent un mélange des deux

Quand utiliser un mock ?

Dépendance	Exemple	Dans les tests
En mémoire	`PriceCalculator`, entités	**réelle**
Hors processus, **gérée**	base de l'application	réelle (intégration) ou **fake**
Hors processus, **non gérée**	SMTP, bus, API de paiement	**mock**

- **Gérée** : seule l'application y accède, ses interactions sont un détail d'implémentation
- **Non gérée** : visible par d'autres systèmes, ses interactions font partie du **contrat**
- — Vladimir Khorikov

Les signes du sur-mocking

- Le setup des mocks est **plus long** que le test
- Le test **répète** l'implémentation, appel par appel
- Un refactoring sans changement de comportement **casse** des tests
- Des mocks qui retournent des mocks
- Tous les tests passent… et l'application est **cassée**
- 💡 Ne mocker que les types **qu'on possède** : envelopper les SDK tiers dans un adaptateur

À retenir

- Tester les **erreurs** comme une sortie : type, message, `rejects` avec `await`
- Préférer **sortie > état > communication**
- Rendre les dépendances **explicites** et injectables
- **Stub** pour les entrées, **mock** pour les effets sortants vers l'extérieur

MODULE 04

Behavior-Driven Development

Implicite et inconnu

> "La livraison est offerte pour les grosses commandes."

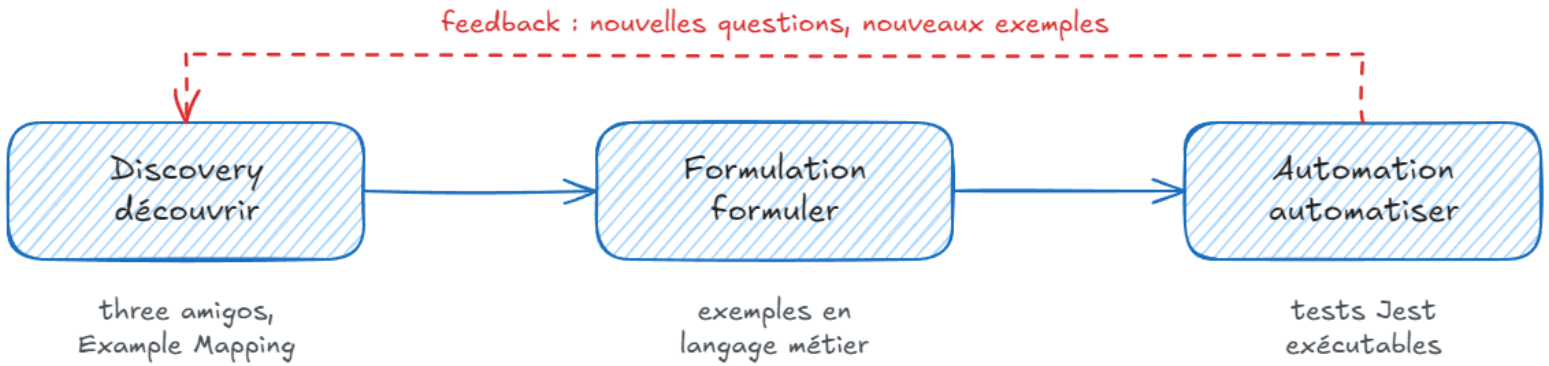
- À partir de **quel montant** ?
- Avant ou après les **remises** ?
- Pour **tous les clients** ?
- Et pour un **panier vide** ?

⚠️ Deux développeurs peuvent écrire des tests verts... et livrer deux comportements **différents**

La démarche BDD

- **Behavior-Driven Development** : Dan North, 2006, issu du TDD
- Une **compréhension partagée** des comportements métier
- Des **exemples** concrets pour lever les ambiguïtés avant de coder
- **Faire évoluer** ensemble logiciel, tests et compréhension

Les trois pratiques



- **Discovery** : comprendre le besoin ensemble, par des exemples
- **Formulation** : écrire les exemples en langage métier, ici dans les noms des tests
- **Automation** : exécuter les exemples comme tests

Gherkin n'est pas le BDD

> « Si vous pensez que Cucumber est un outil de test, lisez la suite : vous vous trompez. »
>
> — Aslak Hellesøy, créateur de Cucumber

- **BDD** : une démarche de **collaboration**, s'accorder sur des exemples avant de coder
- **Gherkin** : un **format texte** (`Étant donné… Quand… Alors…`) pour formuler ces exemples
- **Cucumber**, SpecFlow, Behave : des outils pour **automatiser** des fichiers Gherkin
- Le format et l'outil sont **optionnels**, la conversation ne l'est pas

La conversation d'abord

> « Le BDD, c'est aider les clients, les parties prenantes, à avoir une conversation légèrement structurée avec l'équipe de livraison. »
>
> — Dan North, 2009

Pratique	Résultat
Gherkin écrit seul, pour scripter des tests d'IHM	de l'**automatisation de tests**
Exemples découverts ensemble, en langage métier	du **BDD**, avec ou sans Gherkin

- Dans ce module : exemples, Example Mapping et Jest, **sans** Gherkin

Changer d'état d'esprit

Point de vue	Question	Nom du test
Technique	Quelle **méthode** tester ?	🚫 `shippingCost returns 4.9`
Comportement	Dans cette **situation**, que doit-il se passer ?	✅ `facture 4,90 € pour un panier de 30 €`

- Le test décrit un **comportement**, pas une méthode
- Le mot "test" s'efface au profit de **comportement** et d'**exemple**

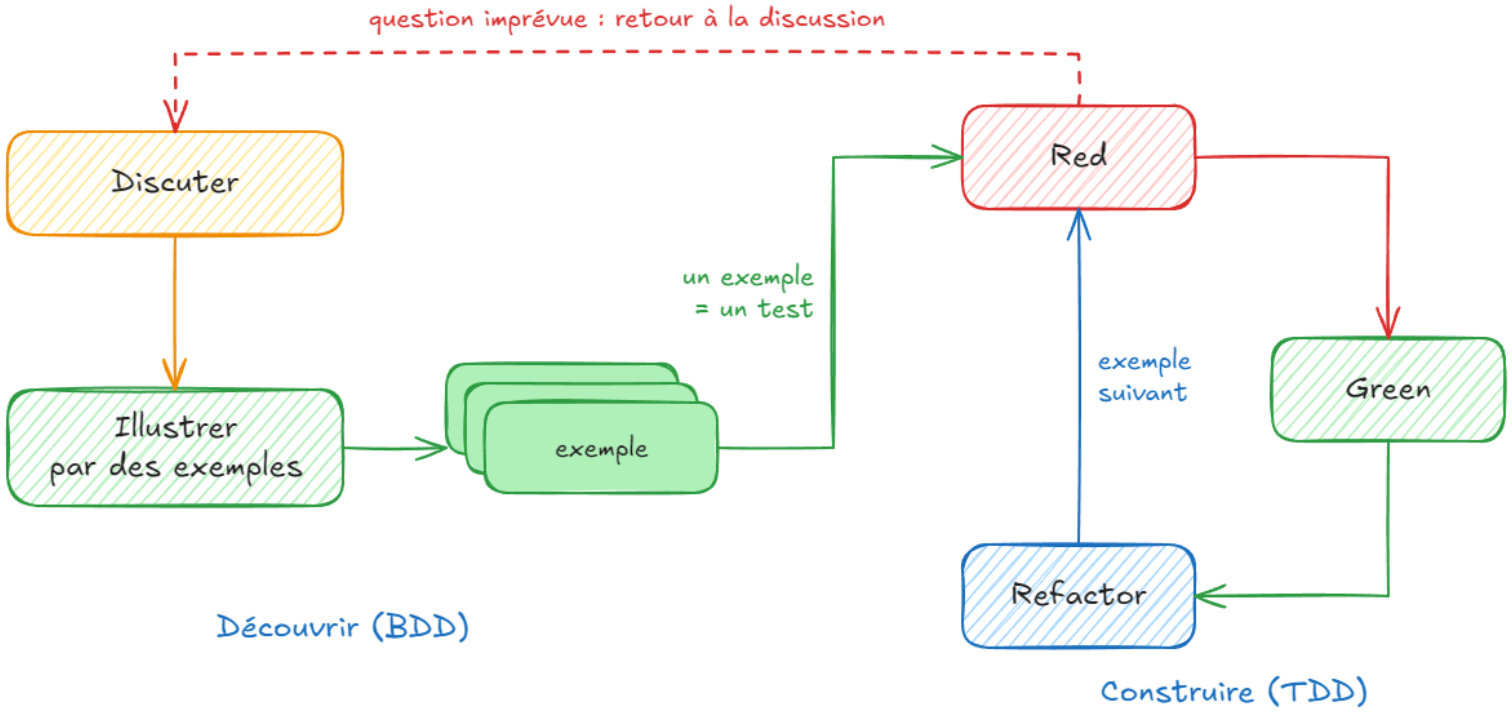
Le comportement, vu de l'extérieur

Un **contexte**, une **action**, un **résultat observable** :

> Pour un client standard dont le panier vaut 30 €, calculer les frais de livraison donne 4,90 €.

- Observable depuis l'**API publique** de l'unité, même sans interface utilisateur
- Une **valeur retournée**, un **changement d'état** ou un **refus**
- Une unité de comportement peut mobiliser **plusieurs objets**
- Le test reste vrai si l'**implémentation** change

BDD et TDD se complètent



- Les exemples disent **quoi** construire ; la boucle TDD, **comment**, par petits pas
- Une question imprévue ramène à la **discussion**

BDD et tests unitaires

- Une **règle métier en mémoire** : le terrain des tests unitaires
- Les exemples deviennent des tests dans le **framework habituel** : Jest, sans outil supplémentaire
- La même démarche guide aussi les **tests d'acceptance**

⚠️ Un test unitaire ne prouve pas que le **parcours complet** fonctionne

Given / When / Then

BDD	AAA	Exemple
Given : étant donné	Arrange	un client standard, un panier de 30 €
When : quand	Act	on calcule les frais de livraison
Then : alors	Assert	ils valent 4,90 €

- Même structure que **AAA**, vocabulaire orienté **comportement**
- Une phrase lisible par le **métier**

Un test orienté comportement

```
describe('Frais de livraison', () => {
  describe('pour un client standard', () => {
    it('facture 4,90 € pour un panier de 30 €', () => {
      const order = { customer: 'standard' as const, total: 30 };

      const cost = shippingCost(order);

      expect(cost).toBe(4.9);
    });
  });
});
```

- `describe` : la **fonctionnalité**, puis le **contexte** (Given)
- `it` : l'**action** et le **résultat** attendus (When / Then)

Des exemples concrets

```
describe('pour un client standard', () => {
  it('facture 4,90 € juste sous le seuil, à 49,99 €', () => {
    expect(shippingCost({ customer: 'standard', total: 49.99 })).toBe(4.9);
  });

  it('offre la livraison à partir de 50 €', () => {
    expect(shippingCost({ customer: 'standard', total: 50 })).toBe(0);
  });
});
```

- **Données réelles** : des montants qui révèlent les **bornes**
- Un test par **exemple**, nommé avec les **mots du métier**

Le rapport comme spécification

```
npx jest --verbose
```

```
Frais de livraison
  pour un client standard
    √ facture 4,90 € pour un panier de 30 €
    √ facture 4,90 € juste sous le seuil, à 49,99 €
    √ offre la livraison à partir de 50 €
  pour un client premium
    √ offre toujours la livraison
```

- Le rapport **se lit** comme les règles métier
- **Documentation vivante** : un écart avec le code rend un test rouge

Écrire des tests orientés comportement

- **Un contexte** par `describe`, **un exemple** par `it`
- Les **mots du domaine** dans les noms et dans le code (ubiquitous language)
- Aucun détail technique dans le nom : ni méthode, ni classe, ni mock
- **L'essentiel** : retirer tout ce qui n'illustre pas la règle
- Pas plus de **2–3 niveaux** de `describe`

Des scénarios BRIEF

Lettre	Dans un test Jest
Business language	les mots du domaine dans `describe`, `it` et le code
Real data	des valeurs concrètes qui révèlent les bornes : 49,99 € / 50 €
Intention revealing	le comportement attendu : ni méthode, ni classe, ni mock
Essential	retirer tout ce qui n'illustre pas la règle
Focused	une seule règle par test
Brief	court : quelques lignes

- Conçu pour les scénarios Gherkin, valable pour tout exemple : tests, cartes 🟩

— Seb Rose et Gáspár Nagy, *The BDD Books – Formulation*

Trois points de vue

Point de vue	Question utile
Métier / produit	Quel résultat apporte de la valeur ?
Développement	Quelles distinctions changent le comportement ?
Test / qualité	Quel exemple contredit notre interprétation ?

- Les **three amigos** : des perspectives, pas forcément trois personnes
- Les **mêmes mots** dans la conversation et dans les tests
- Une incertitude devient une **question explicite**, pas une règle inventée

Règle, exemple, question

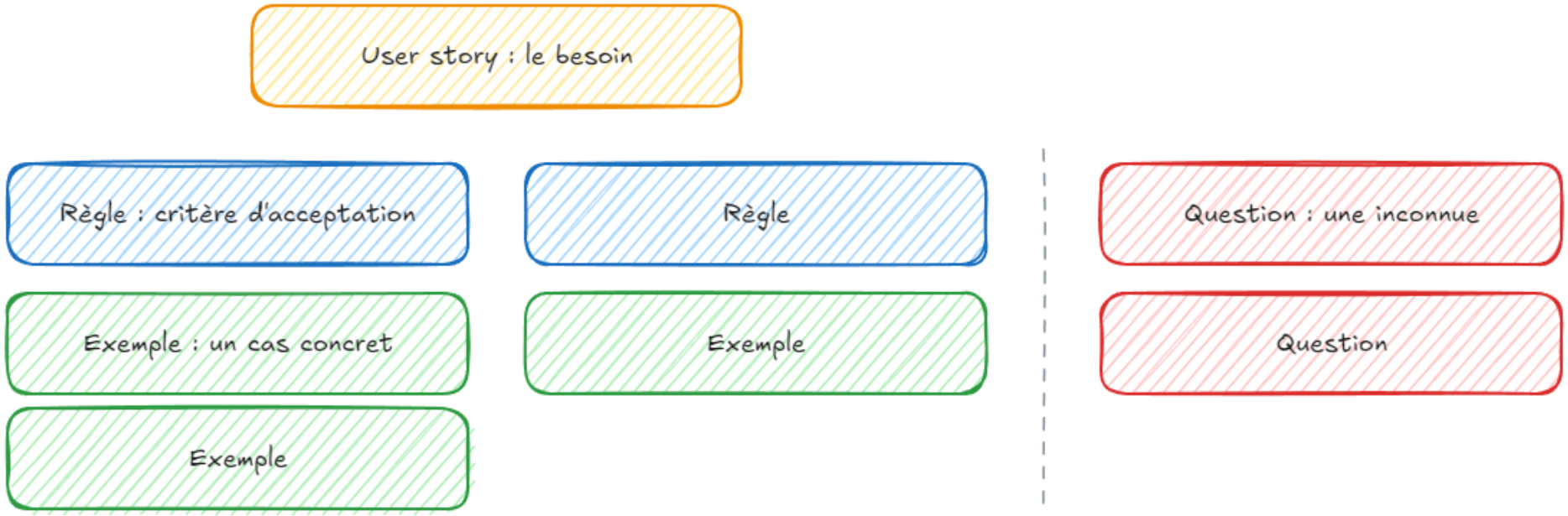
Élément	Illustration
Besoin	Connaître les frais de livraison avant de commander
Règle	Un client standard paie 4,90 € sous 50 € d'achat
Exemple	Client standard, panier de 30 € → frais de 4,90 €
Question	Le seuil s'applique-t-il avant ou après remise ?

- Une règle **généralise** ; un exemple donne des **valeurs concrètes**
- Sans résultat attendu partagé, il reste une **question**

L'atelier Example Mapping

- Proposé par **Matt Wynne** (Cucumber), 2015
- Atelier court : environ **25 minutes** par user story
- Les **three amigos** autour d'une table, des cartes de **4 couleurs**
- But : découvrir les **règles** et les **exemples** avant de coder
- Résultat : une story **prête** ou des **questions** à lever

Les quatre cartes



- Chaque **règle** forme une colonne avec ses **exemples** ; les **questions** restent à part

Déroulé de l'atelier

1. Poser la 🟨 **story** et la lire à voix haute
2. Énoncer les 🟦 **règles** connues, une carte par règle
3. Illustrer chaque règle par des 🟩 **exemples** concrets
4. Noter chaque doute sur une 🟥 **question**, sans bloquer l'atelier
5. Une question tranchée devient une **règle** ou un **exemple**
6. Conclure : story prête, à découper ou à reporter ?

Lire la table

- Beaucoup de 🟥 : la story n'est **pas prête**
- Beaucoup de 🟦 : la story est **trop grosse**, la découper
- Une 🟦 sans 🟩 : règle encore **floue**
- 🟩 → les **tests** ; 🟦 → les `describe`
- 🟥 restantes → à poser à la **bonne personne** avant de coder

Kata Potter : le besoin

- Une librairie vend les **5 tomes** d'une série, **8 €** l'unité
- Une remise s'applique à chaque lot de tomes **différents** :

Tomes différents	2	3	4	5
Remise	5 %	10 %	20 %	25 %

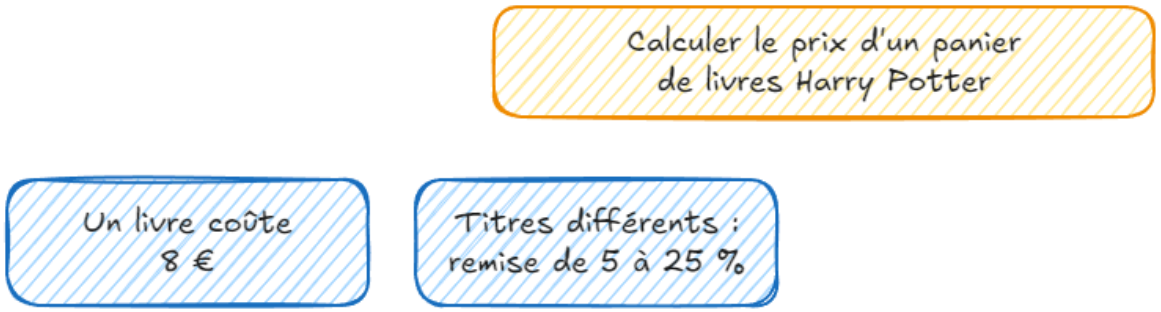
- Objectif : calculer le **prix d'un panier**
- Point de départ : cet énoncé, avec ses **zones d'ombre**

Example Mapping : Kata Potter



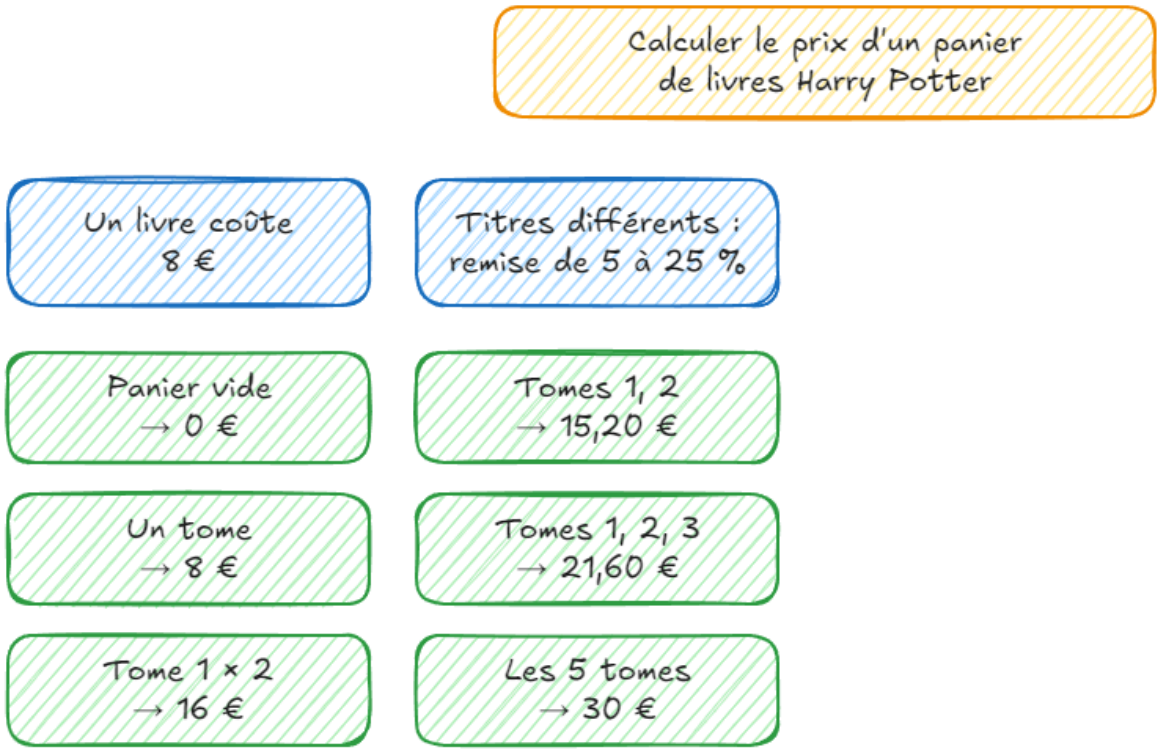
- La **story** : le besoin, en une phrase

Example Mapping : Kata Potter



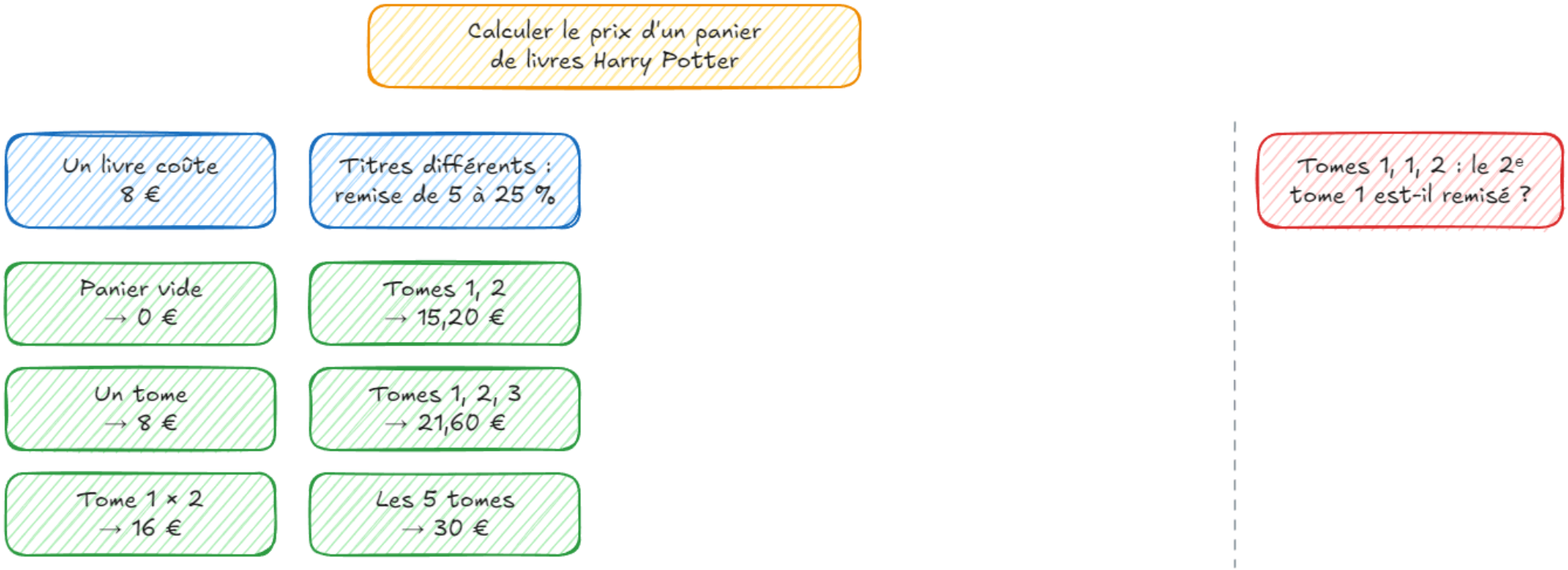
- 🟦 Les **règles** évidentes, énoncées par le métier

Example Mapping : Kata Potter



- 🟩 Des **exemples** concrets, calculés ensemble : 2 × 8 € × 0,95 = 15,20 €

Example Mapping : Kata Potter



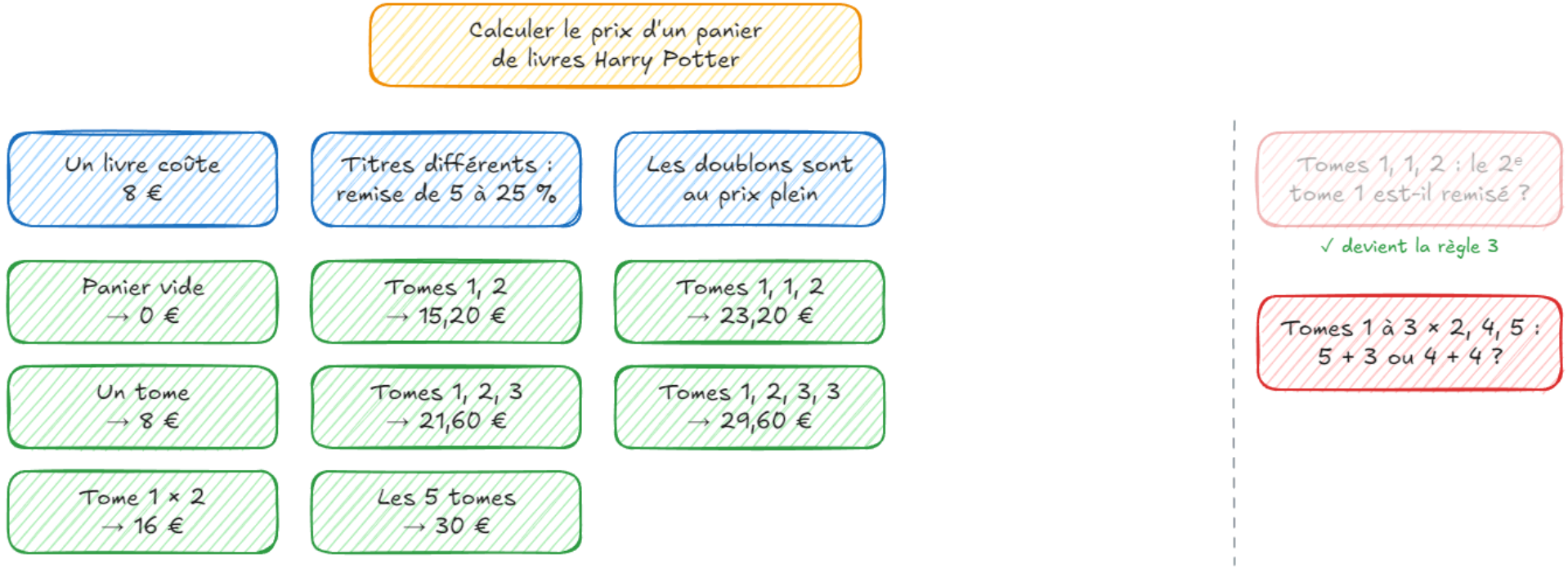
- Un panier avec un **doublon** : l'énoncé ne dit pas s'il est remisé

Example Mapping : Kata Potter



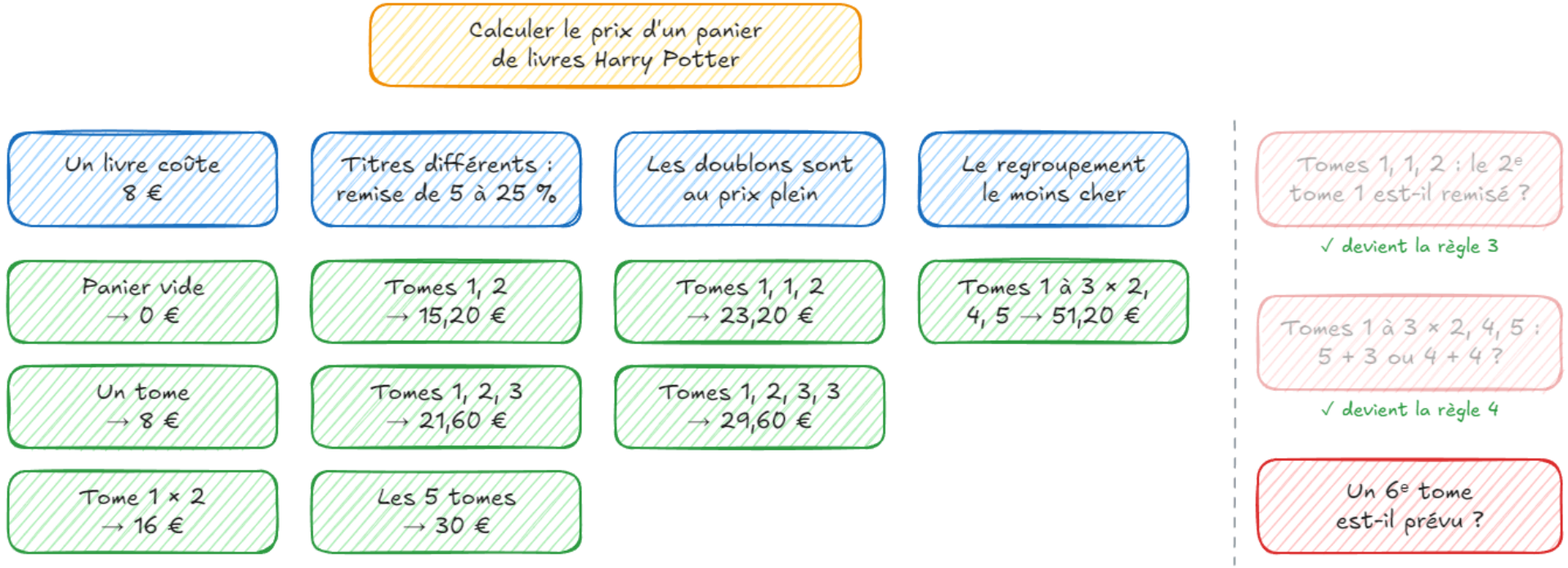
- La réponse du métier devient une **règle**, avec ses exemples : 15,20 € + 8 € = 23,20 €

Example Mapping : Kata Potter



- Le testeur cherche un **contre-exemple** : deux regroupements possibles
- 5 + 3 : 30 € + 21,60 € = **51,60 €** ; 4 + 4 : 25,60 € × 2 = **51,20 €**

Example Mapping : Kata Potter



- Le métier tranche : le regroupement **le moins cher** pour le client
- Une question reste **ouverte** : elle ne bloque pas la story

De la carte aux tests

```
describe("Prix d'un panier de livres", () => {
  describe('un livre coûte 8 €', () => {
    it.todo('panier vide : 0 €');
    it.todo('un tome : 8 €');
  });

  describe('les doublons sont au prix plein', () => {
    it.todo('tomes 1, 1 et 2 : 23,20 €');
  });

  describe('le regroupement le moins cher est retenu', () => {
    it.todo('tomes 1 à 3 en double, tomes 4 et 5 : 51,20 €');
  });
});
```

- 🟦 → `describe`, 🟩 → `it` ; `it.todo` : la liste de tests, implémentée ensuite en TDD

À retenir

- **BDD** : une compréhension partagée du comportement, par des **exemples** concrets
- Un test unitaire BDD : **contexte**, **action**, **résultat observable**, en langage métier
- **Example Mapping** : story, règles, exemples, questions, en 25 minutes
- Les exemples deviennent la **liste de tests** du TDD

MODULE 05

Couverture, intégration continue, pièges et synthèse

Mesurer la couverture

- **Couverture de code** : part du code exécutée par les tests
- Mesurée par instrumentation pendant l'exécution
- Révèle le code **jamais exécuté** par un test
- Ne dit **rien** sur la qualité des assertions

```
npx jest --coverage
```

Les métriques

Métrique	Question posée
Statements	chaque instruction a-t-elle été exécutée ?
Branches	chaque branche (`if` , `?` `:` , `&&` , `` ` ``
Functions	chaque fonction a-t-elle été appelée ?
Lines	chaque ligne a-t-elle été exécutée ?

- **Branches** : la plus exigeante des quatre
- Rapport HTML détaillé dans `coverage/lcov-report/index.html`

Le rapport Jest

```ts
export function shippingCost(total: number, isPremium: boolean): number {
  if (isPremium || total >= 50) {
    return 0;
  }
  return 4.9;
}
```

```
-------------|---------|----------|---------|---------|-------------------
File         | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-------------|---------|----------|---------|---------|-------------------
 shipping.ts |     100 |      100 |     100 |     100 |
-------------|---------|----------|---------|---------|-------------------
```

- Testé avec un client premium et un panier de 20 € : **100 %**
- Le seuil de **50 €** n'a pourtant jamais été vérifié

Un seuil minimal

```
// jest.config.js
module.exports = {
  preset: 'ts-jest',
  collectCoverageFrom: ['src/**/*.ts', '!src/**/*.test.ts'],
  coverageThreshold: {
    global: { branches: 80, functions: 80, lines: 80, statements: 80 },
  },
};
```

- `jest --coverage` **échoue** si le seuil n'est pas atteint
- `collectCoverageFrom` : compter aussi les fichiers **jamais importés** par un test
- Un **plancher** contre la régression, pas un objectif

Complexité cyclomatique

- Nombre de **chemins indépendants** dans une fonction (McCabe, 1976)
- 1 + nombre de décisions (`if` , `case` , `&&` , `||` , boucles…)
- **Minimum** de tests pour couvrir toutes les branches
- Au-delà de **10** : fonction difficile à comprendre et à tester
- La limiter avec ESLint : `complexity: ['error', 10]`

L'illusion du 100 %

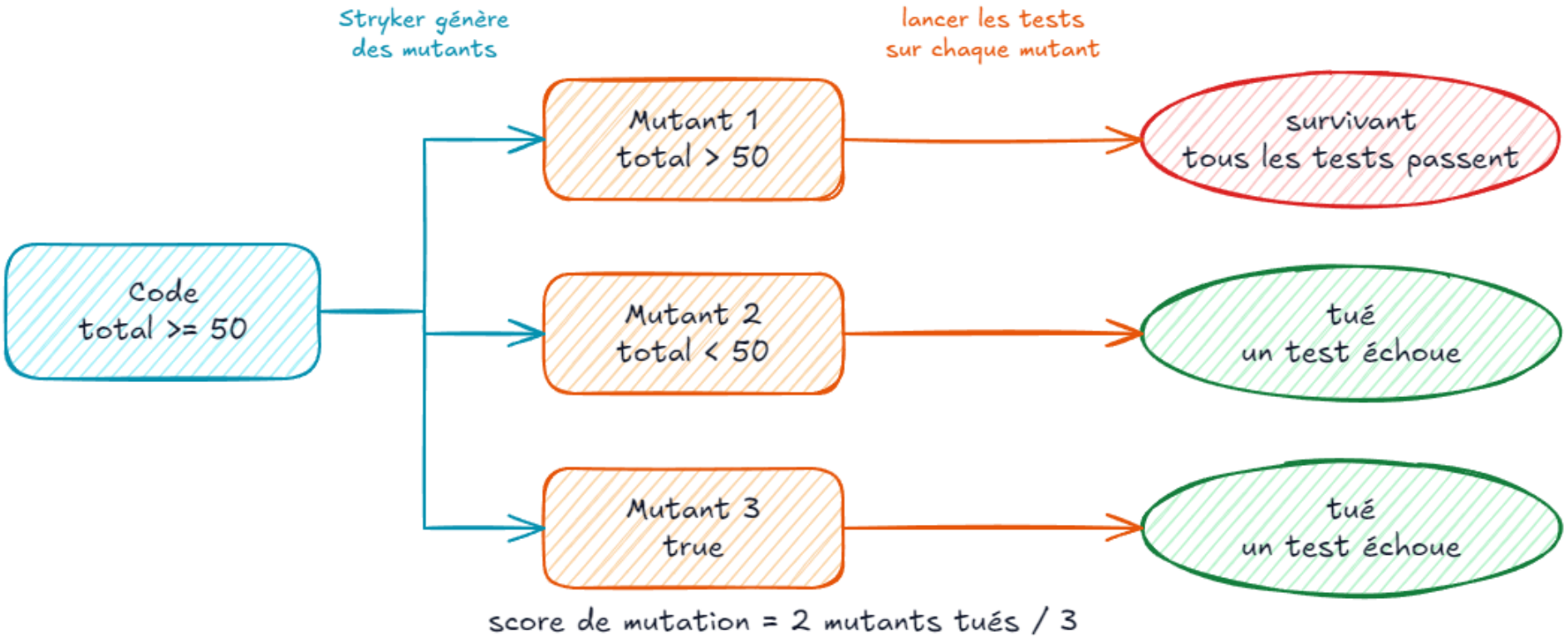
```
it('calcule les frais de livraison', () => {
  shippingCost(20, false);
  shippingCost(20, true);
});
```

- **100 %** de couverture… et **aucune** assertion
- La couverture mesure ce qui est **exécuté**, pas ce qui est **vérifié**
- Le test ne peut **jamais** échouer : c'est un *Liar*

Ce que dit la couverture

- 🔻 Couverture **faible** : signal fiable, du code n'est pas testé
- 🔺 Couverture **élevée** : ne prouve **rien** sur la qualité des tests
- 🎯 En faire un **objectif** pousse aux tests sans valeur (loi de Goodhart)
- 🔍 L'utiliser comme **outil d'exploration** : quelles zones à risque ne sont pas testées ?
- ✅ Viser une couverture élevée **du code métier**, pas de la plomberie

Le mutation testing



Le mutation testing

- Introduire de petits **bugs** (mutants) dans le code de production
- Relancer les tests pour **chaque** mutant
- **Tué** : au moins un test échoue, les tests protègent ce code
- **Survivant** : tous les tests passent, la modification n'est pas détectée
- **Score de mutation** = mutants tués / mutants générés
- Mesure la **qualité** des tests, là où la couverture mesure leur étendue

Stryker

```
npm install --save-dev @stryker-mutator/core @stryker-mutator/jest-runner
npx stryker init
npx stryker run
```

```
{
  "testRunner": "jest",
  "mutate": ["src/**/*.ts", "!src/**/*.test.ts"],
  "reporters": ["clear-text", "html"],
  "coverageAnalysis": "perTest"
}
```

- Équivalents : **PIT** (Java), **mutmut** (Python), Stryker.NET (C#)

Des mutants survivants

```
[Survived] EqualityOperator
src/shipping.ts:2:20
-     if (isPremium || total >= 50) {
+     if (isPremium || total > 50) {
Tests ran:
    shippingCost est de 4,90 € en dessous de 50 €

-------------|------------------|----------|-----------|------------|
             | % Mutation score |          |           |            |
File         |  total | covered | # killed | # survived | # no cov |
-------------|--------|---------|----------|------------|----------|
 shipping.ts |  73.68 |   73.68 |       14 |          5 |        0 |
```

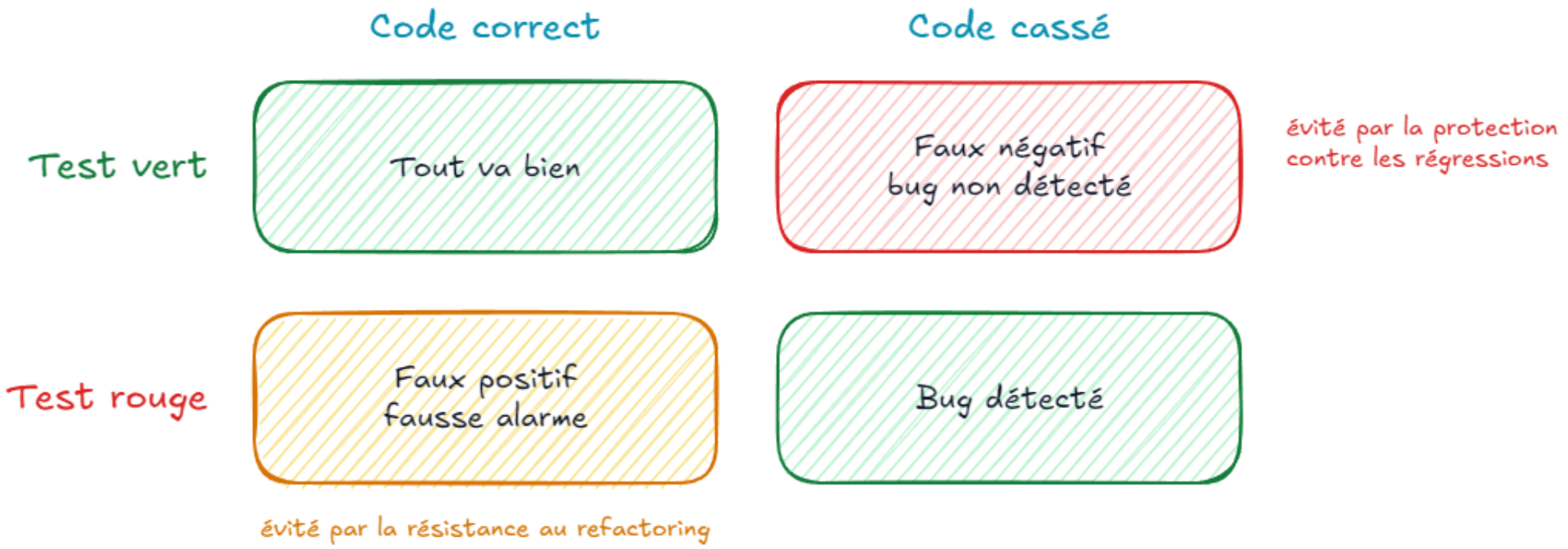
- **100 %** de couverture, mais **73 %** de score de mutation
- Le mutant `>=` → `>` survit : la **borne** de 50 € n'est pas testée

Tuer le mutant

```
it.each([
  [50, 0],
  [49.99, 4.9],
])('un panier de %d € coûte %d € de livraison', (total, expected) => {
  expect(shippingCost(total, false)).toBe(expected);
});
```

- Chaque survivant pointe un **cas manquant**, souvent une **borne**
- Mutation testing : lent, à lancer sur le **code critique** ou en **incrémental**
- Certains mutants sont **équivalents** (même comportement) : les ignorer

Faux positifs, faux négatifs



Faux positifs, faux négatifs

- **Faux positif** : le test échoue alors que le code fonctionne
  - fausse alarme, souvent après un refactoring ; la confiance dans les tests s'érode
- **Faux négatif** : le test passe alors que le code est cassé
  - bug non détecté ; assertions faibles ou absentes
- Protection contre les régressions ↘ faux négatifs
- Résistance au refactoring ↘ faux positifs
- ⚠️ Le vocabulaire varie : préciser « fausse alarme » ou « bug non détecté »

Les tests fragiles

- Un test **fragile** casse sans que le comportement ait changé
- Cause principale : le **couplage à l'implémentation**
  - vérifier des appels internes, l'ordre des appels, des détails de structure
  - tester des méthodes privées
  - recopier l'algorithme dans le test
- Symptôme : chaque refactoring **réécrit** les tests
- Remède : vérifier le **résultat observable**, pas le chemin pour l'obtenir

Couplé à l'implémentation

```
// ❌ Vérifie COMMENT le prix est calculé
it('calcule le total', () => {
  const calculator = new PriceCalculator();
  const applyDiscount = jest.spyOn(calculator as any, 'applyDiscount');

  calculator.total([{ label: 'Clavier', price: 120 }]);

  expect(applyDiscount).toHaveBeenCalledWith(120, 0.1);
});
```

```
// ✅ Vérifie CE QUE fait le calcul
it('applique 10 % de remise au-delà de 100 €', () => {
  expect(new PriceCalculator().total([{ label: 'Clavier', price: 120 }])).toBe(108);
});
```

Et les méthodes privées ?

- **Ne pas** les tester directement : ce sont des détails d'implémentation
- Les couvrir via l'**API publique** qui les utilise
- Méthode privée trop complexe pour être testée ainsi : signe d'une **classe cachée**
  - l'**extraire** dans sa propre classe, avec une API publique testable
- ❌ Ne pas rendre public « juste pour le test »
- ❌ Pas de `as any` pour atteindre un membre privé

Surspécification

```
// ❌ Fige toute la structure : un nouveau champ casse le test
expect(invoice).toEqual({
  id: 'F-2026-001', createdAt: new Date('2026-01-01'), currency: 'EUR',
  lines: [{ label: 'Clavier', price: 120 }], total: 108, shipping: 0,
});
```

```
// ✅ Vérifie ce que le test raconte
expect(invoice).toMatchObject({ total: 108, shipping: 0 });
```

- Un test = **un comportement** : n'asserter que ce qui le décrit
- Même piège avec les **snapshots** géants validés sans les lire

Les tests instables

Cause	Remède
Heure, date	horloge injectée, fake timers
Hasard	générateur injecté, `spyOn(Math, 'random')`
Ordre des tests, état partagé	état recréé dans chaque test
Promesse non attendue	`await` , `expect.assertions(n)`
Réseau, base, fichiers	doublures en test unitaire

- **Flaky test** : passe ou échoue sans changement de code
- Un test instable **ignoré** une fois l'est ensuite toujours : le corriger ou le supprimer

État partagé

```
// ❌ Le panier est partagé : le résultat dépend de l'ordre des tests
const cart = new Cart([]);

it('ajoute un article', () => {
  cart.add({ label: 'Clavier', price: 120 });
  expect(cart.itemCount()).toBe(1);
});

it('démarre vide', () => {
  expect(cart.itemCount()).toBe(0);
});
```

- Créer l'objet **dans** chaque test (ou dans un `beforeEach`)
- `jest --randomize` : mélanger l'ordre pour révéler ces dépendances

Anti-patterns

Anti-pattern	Symptôme	Remède
Excessive setup	des dizaines de lignes d'Arrange	builders, revoir la conception
The Giant	un test vérifie tout	un comportement par test
The Liar	ne peut jamais échouer	voir le test **rouge** au moins une fois
The Inspector	viole l'encapsulation	tester via l'API publique
The Slow Poke	prend des secondes	doublures, isoler les I/O

The Liar

```
// ❌ L'événement est émis après la fin du test : l'assertion ne s'exécute jamais
it('notifie le client', () => {
  orderEvents.on('confirmed', (order) => expect(order.status).toBe('confirmed'));

  checkout.confirm(cart);
});
```

```
// ✅ Le test attend l'événement et échoue si l'assertion n'est pas exécutée
it('notifie le client', async () => {
  expect.assertions(1);
  const confirmed = new Promise<Order>((resolve) => orderEvents.once('confirmed', resolve));

  await checkout.confirm(cart);

  expect((await confirmed).status).toBe('confirmed');
});
```

Le code legacy

> Le code legacy, c'est du code sans tests.
>
> — Michael Feathers, *Working Effectively with Legacy Code*

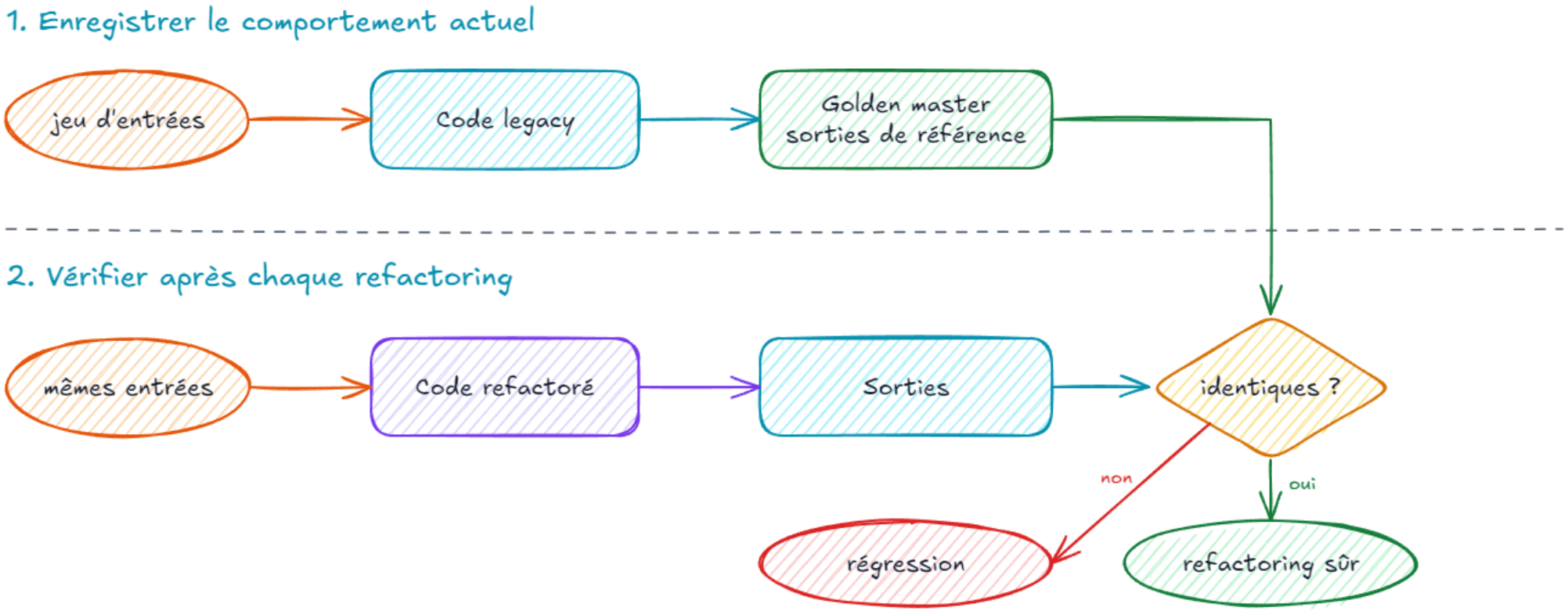
- Le **dilemme** : pour modifier en sécurité, il faut des tests…
- … et pour ajouter des tests, il faut souvent modifier le code
- Avancer par **petites étapes sûres**, refactorings automatisés de l'IDE

Modifier du legacy

1. Identifier les **points de changement**
2. Trouver les **points de test** : où observer le comportement ?
3. **Casser les dépendances** qui empêchent de tester
4. Écrire des **tests de caractérisation**
5. **Modifier** et **refactorer**, en TDD pour le nouveau code

— Michael Feathers

Golden master



Golden master

- **Test de caractérisation** : figer le comportement **actuel**, bugs compris
- Générer un **grand jeu d'entrées** couvrant le code à refactorer
- **Enregistrer** les sorties produites par le code existant (le golden master)
- Vérifier que chaque refactoring produit **exactement** les mêmes sorties
- Remplacer progressivement par de vrais tests unitaires, puis **supprimer** le golden master

Approval testing avec Jest

```
const types = ['jour', 'nuit'];
const ages = [0, 5, 6, 17, 18, 64, 65];
const days = ['lundi', 'samedi'];

it('caractérise le calcul de prix existant', () => {
  const results = types.flatMap((type) =>
    ages.flatMap((age) =>
      days.map((day) => `${type} | ${age} ans | ${day} → ${legacyPrice(type, age, day)}`),
    ),
  );

  expect(results).toMatchSnapshot();
});
```

- 1er lancement : écrit `__snapshots__/*.snap` ; ensuite : compare
- Combinaisons d'entrées → **couverture** et **mutation testing** pour valider le filet

Snapshots : bon usage

- Le snapshot est **relu** et commité comme du code
- `jest -u` met à jour : seulement pour un changement **voulu**
- Idéal pour le **legacy** et les sorties volumineuses (rapports, sérialisation)
- ⚠️ Pas pour remplacer des assertions ciblées en TDD
- ⚠️ Snapshot géant = personne ne le lit = test qui valide tout

Casser les dépendances

- **Seam** (couture) : endroit où changer un comportement **sans** modifier le code à cet endroit
- **Paramètre** : passer la dépendance au lieu de la créer
- **Extract and override** : extraire l'appel dans une méthode, la surcharger en test
- **Extraire une interface** : injecter une doublure
- Les techniques du jour 2, appliquées **avec prudence** sur du code non testé

Sprout et wrap

- **Sprout method / class** : écrire le **nouveau** code à part, en TDD, puis l'appeler depuis le legacy
- **Wrap method / class** : envelopper l'existant pour ajouter un comportement avant ou après
- Le legacy reste **intact** ; le nouveau code est **testé**

```
export function legacyCheckout(cart: LegacyCart): number {
  // … 200 lignes non testées …
  const total = computeLegacyTotal(cart);
  return applyLoyaltyDiscount(total, cart.customer); // 🌱 nouveau, testé en TDD
}
```

Parallel change

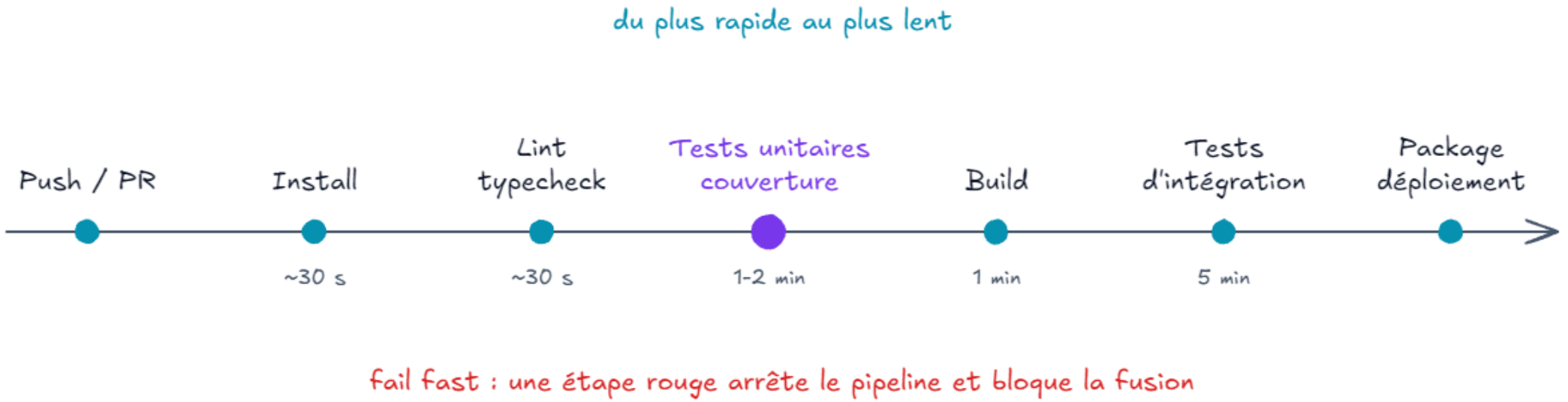


- Changer une interface **sans tout casser** d'un coup
- Ancienne et nouvelle versions **coexistent** pendant la migration
- Les tests restent **verts** à chaque étape ; chaque étape peut être commitée

L'intégration continue

- **Intégrer** son code à la branche principale **souvent** (au moins une fois par jour)
- Chaque intégration est **vérifiée automatiquement** : build, tests, qualité
- 🔔 Régressions détectées en **minutes**, pas en recette
- 🤝 Une branche principale **toujours** dans un état livrable
- 📈 Métriques suivies dans le temps

Le pipeline



- Les tests unitaires tournent **tôt** : retour en quelques minutes

Quality gates

- **Tests** : 100 % de tests verts, sans exception
- **Couverture** : seuil minimal (`coverageThreshold`), ou pas de baisse
- **Analyse statique** : lint, typecheck, SonarQube (*clean as you code*)
- **Mutation** : score minimal sur le code critique (Stryker `thresholds.break`)
- Rendues **obligatoires** : branch protection sur la PR
- Appliquer d'abord au **nouveau code** plutôt qu'à tout le legacy

Garder un feedback rapide

- **Cache** des dépendances
- **Paralléliser**
- **Lancer d'abord** les tests impactés :
- **Séparer** tests unitaires (chaque push) et tests lents (merge, nuit)
- Objectif : un retour en **moins de 10 minutes**

Les outils de CI

Outil	Fichier	Particularité
GitHub Actions	`.github/workflows/*.yml`	intégré à GitHub, marketplace
GitLab CI	`.gitlab-ci.yml`	rapports JUnit et couverture dans la MR
Jenkins	`Jenkinsfile`	auto-hébergé, très extensible
Azure DevOps	`azure-pipelines.yml`	écosystème Microsoft

- Mêmes **étapes**, mêmes **principes** : seule la syntaxe change

Par langage

	Java	Python	JS / TS
Rapport de tests	Surefire (JUnit XML)	`pytest --junitxml`	jest-junit
Couverture	JaCoCo	coverage.py, pytest-cov	Istanbul, V8
Mutation	PIT	mutmut	Stryker
Analyse statique	SonarQube, SpotBugs	SonarQube, ruff	SonarQube, ESLint

- Les **concepts** sont identiques : seuls les outils changent

À retenir

- 100 % de couverture ne prouve **rien** : vérifier la qualité des tests (mutation)
- Tester le **comportement observable**, jamais l'implémentation
- Corriger ou supprimer les tests **instables**, ne pas les ignorer
- Sur du **legacy** : golden master d'abord, refactoring ensuite
- Une **CI** rapide et bloquante rend les tests utiles au quotidien